TP2 - File Integrity Checker and IOC Matcher in Rust

Module 7.1 - Programming with Rust

Student: Abderahman Mohamed Lemin

Student ID: 25235

Date: June 15, 2026

GitHub: https://github.com/x0abderahman/tp2_integrity_checker

Defensive Security - File Integrity Verification Tool

1. Objective

This project implements a command-line File Integrity Checker and IOC (Indicator of Compromise) Matcher written in Rust. The tool scans a file or directory, calculates SHA-256 cryptographic hashes for each regular file, compares them against a provided list of known malicious hashes (IOCs), and produces a structured CSV report.

File integrity checking is a fundamental defensive security technique. It allows security analysts to verify that files have not been tampered with by comparing their current cryptographic hash against a known good or known bad baseline. An IOC (Indicator of Compromise) is a piece of forensic evidence - such as a file hash, IP address, or domain name - that indicates a potential security breach.

The tool is designed with secure programming practices: it avoids unsafe code, handles errors gracefully without panicking, and splits functionality into well-defined modules.

2. Environment

The development environment uses a Docker Compose setup with Debian Bookworm:

```
mkdir -p workspace
docker compose up -d --build
docker compose exec rustlab bash
cd /workspace/tp2_integrity_checker
```

Rust toolchain versions:

```
rustc 1.95.0 (59807616e 2026-04-14)
cargo 1.95.0 (f2d3ce0bd 2026-03-21)
rustfmt 1.9.0-stable
clippy 0.1.95
```

Project path inside the container: /workspace/tp2_integrity_checker

Student ID: 25235

3. Implementation

3.1 Hashing Module (hashing.rs)

The hashing module implements the SHA-256 hash calculation using the sha2 crate. Files are read in 8KB buffers to handle large files efficiently without loading the entire content into memory. The hash is returned as a lowercase hexadecimal string.

```
pub fn hash_file_sha256(path: &Path) -> Result<String, io::Error> {
    let mut file = File::open(path)?;
    let mut hasher = Sha256::new();
    let mut buffer = [0u8; 8192];
    loop {
        let bytes_read = file.read(&mut buffer)?;
        if bytes_read == 0 { break; }
        hasher.update(&buffer[..bytes_read]);
    }
    Ok(format!("{:x}", hasher.finalize()))
}
```

3.2 IOC Parsing Module (ioc.rs)

The IOC module parses a CSV-like file containing hashes and labels. It handles: empty lines (ignored), comment lines starting with # (ignored), invalid SHA-256 strings (counted but skipped), and valid entries (parsed into IocEntry structs). The is_valid_sha256 function validates that strings are exactly 64 hexadecimal characters.

3.3 Scanner Module (scanner.rs)

The scanner module walks through a target path using the walkdir crate for recursive directory traversal. It scans all regular files in nested directories. Hashing is parallelized using the rayon crate (par_iter). Each file's hash is compared against the IOC list and optionally against a known-good manifest.

3.4 Report Module (report.rs)

The report module generates both CSV and JSON output. The CSV uses the csv crate with columns: path, sha256, status, label. The JSON output includes a complete scan report with timestamp, summary statistics, and per-file details. Results are sorted by path for reproducible output.

3.5 Manifest Module (manifest.rs)

A new module for parsing known-good manifest files. This allows the tool to identify and label trusted files, distinguishing them from unknown or suspicious files. The format is identical to the IOC file format.

3.6 CLI Handling (main.rs)

Command-line arguments support --target, --ioc, --report (required), plus optional --json <PATH>, --only-matches, and --good-manifest <PATH>. Missing arguments produce a clear usage message. The main function delegates to run() which returns Result, avoiding unwrap() in the execution path.

```
Usage:
  tp2_integrity_checker --target <FILE_OR_DIR> --ioc <IOC_FILE> --report <REPORT>
  [--json <JSON>] [--only-matches] [--good-manifest <MANIFEST>]
```

4. Execution Evidence

The following screenshots demonstrate successful execution of the tool:

Figure 2: SHA-256 verification and cargo run output

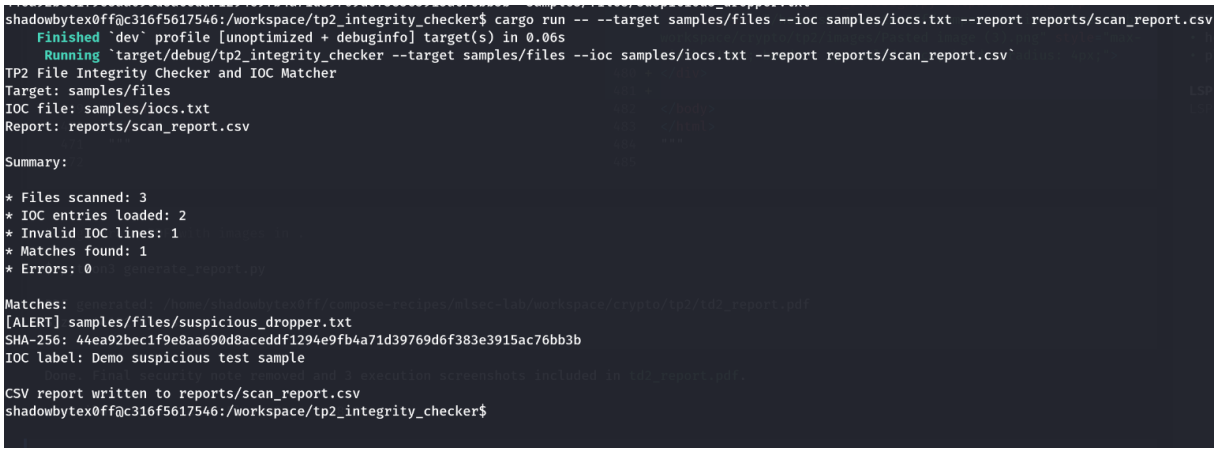

Figure 3: Scan result with match detection

Figure 4: cargo test - 12 unit tests + 4 integration tests

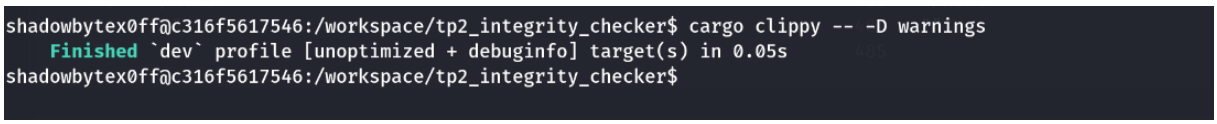

Figure 5: cargo clippy - zero warnings

Generated CSV report content:

```
path,sha256,status,label
samples/files/clean_readme.txt,70bbeaa0f2d408a45827aa9d5bd58209564a5bd7b61d6c069267c9e9e35f97cd,CLEAN,
samples/files/suspicious_dropper.txt,44ea92bec1f9e8aa690d8aceddf1294e9fb4a71d39769d6f383e3915ac76bb3b,MATCH,Dem
o suspicious test sample
samples/files/notes.txt,1888673b4c962129e57b54b81fda2f967c52f87c28e9d7908e5fba0dfed097e3,CLEAN,
```

Test results: 12 unit tests + 4 integration tests = 16 total, all passing.

cargo clippy: zero warnings with -D warnings.

5. Discussion

5.1 Invalid IOC Handling

The IOC file parser gracefully handles invalid lines: lines that are not 64-character hexadecimal strings are counted as invalid and skipped. The program continues execution and reports the number of invalid lines in the summary.

5.2 Missing File Handling

If the target file or directory does not exist, the program prints a clear error message and exits with a non-zero status code.

5.3 Why SHA-256 Over MD5 or SHA-1

SHA-256 is preferred over MD5 and SHA-1 because both are cryptographically broken. SHA-256 remains collision-resistant and is the NIST standard.

5.4 Bonus Features Implemented

The following bonus features have been implemented:

- Recursive directory scanning using the walkdir crate
- JSON output option (--json) with structured scan reports
- --only-matches flag to filter output to suspicious files only
- Known-good manifest mode (--good-manifest) using a separate hash allowlist
- Parallel scanning using rayon threads for multi-core performance
- Integration tests in tests/integration_test.rs (4 test cases)

6. Conclusion

This TP successfully implemented a File Integrity Checker and IOC Matcher in Rust with all mandatory requirements and multiple bonus features.

Rust concepts practiced:
- Multi-module project structure (6 modules)
- External crate dependencies (sha2, csv, walkdir, serde, serde_json, rayon)
- Result and Option for error handling without panics
- Struct definitions, enum types, trait derivations
- File I/O, directory traversal, cryptographic hashing
- Unit testing (12 tests) and integration testing (4 tests)
- Parallel processing with rayon
- Serializable data structures with serde
- Code formatting (cargo fmt) and linting (cargo clippy -D warnings)